VISUALIZATION

Data visualization means showing data in the form of charts or graphs . It helps people see patterns, trends, and insights in data quickly.

Makes complex data easier to understand , Helps in identifying trends, outliers, or patterns,

Used to communicate data clearly to others, Helps in making data-driven decisions.

MATPLOTLIB

Matplotlib, introduced in 2002 by John Hunter, is a comprehensive library for creating static, animated, and interactive 2D plots in Python. It is built on NumPy and works seamlessly with other scientific libraries.

A Matplotlib Plot Contains:

1. Figure – The overall container for a plot (can contain multiple subplots).

2. Axes – The actual plotting area (a figure can contain multiple axes).

3.Axis – Manages ticks and limits on the X and Y axes.

4.Artists – Everything visible on the plot (lines, text, ticks, labels, etc.).

PYPLOT

pyplot is a sub-library of matplotlib and provides a simple interface similar to MATLAB.

It contains functions to create all major chart types: line plots, bar charts, histograms, pie charts, scatter plots, etc.

We typically import it like this:

```
import matplotlib.pyplot as plt
```

We also import numpy as a support for the matplotlib :

```
import numpy as np
```

Some of the plots in matplotlib:

LINE CHART

A line chart connects data points with a straight line.

Used to show trends over time or ordered categories.

✅ **Code Snippet:**

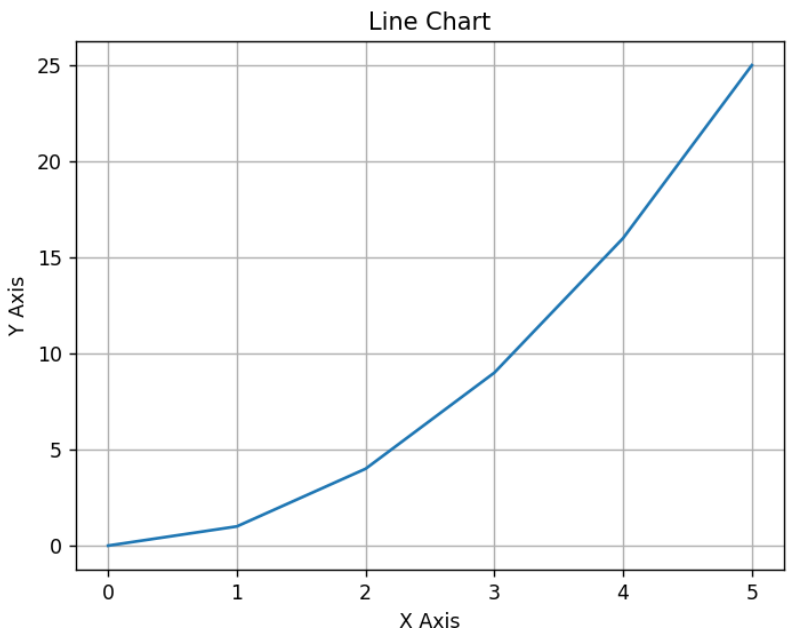
Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np

# Data
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 1, 4, 9, 16, 25])

# Plotting
plt.plot(x, y)
plt.title("Line Chart")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.grid(True)
plt.show()
```

Output:



Description:

np.array() creates arrays.

plot() plots the line chart.

title(), xlabel(), ylabel() add chart details.

show() displays the figure.

BAR GRAPH

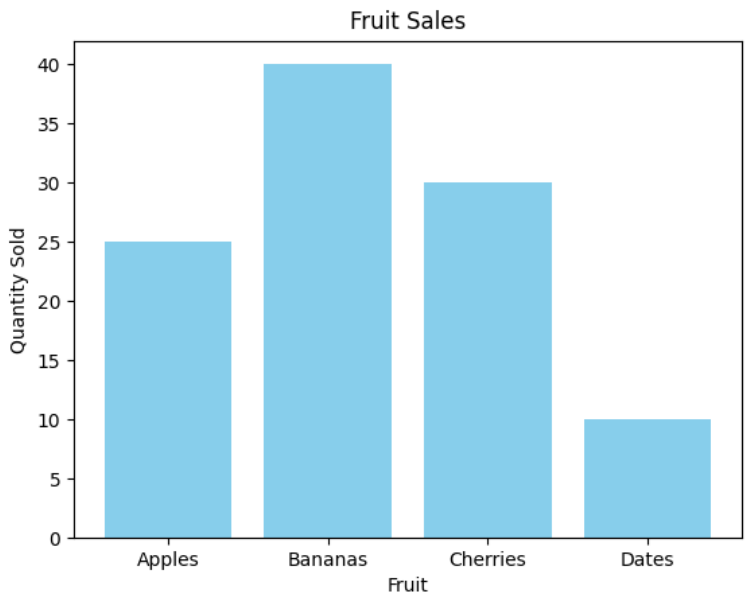
Represents categorical data using rectangular bars. Height of the bar is proportional to the data value

Code snippet:

C: > Users > kumar > shadow.py > ...

```python
import matplotlib.pyplot as plt
categories = ['Apples', 'Bananas', 'Cherries', 'Dates']
values = [25, 40, 30, 10]
plt.bar(categories, values, color='skyblue')
plt.title('Fruit Sales')
plt.xlabel('Fruit')
plt.ylabel('Quantity Sold')
plt.show()
```

Output:



Description:

plt.bar() creates a vertical bar chart.

categories is used for the x-axis labels.

values is used for the heights of the bars.

color='skyblue' makes it more visually appealing (optional).

plt.show() displays the plot window.

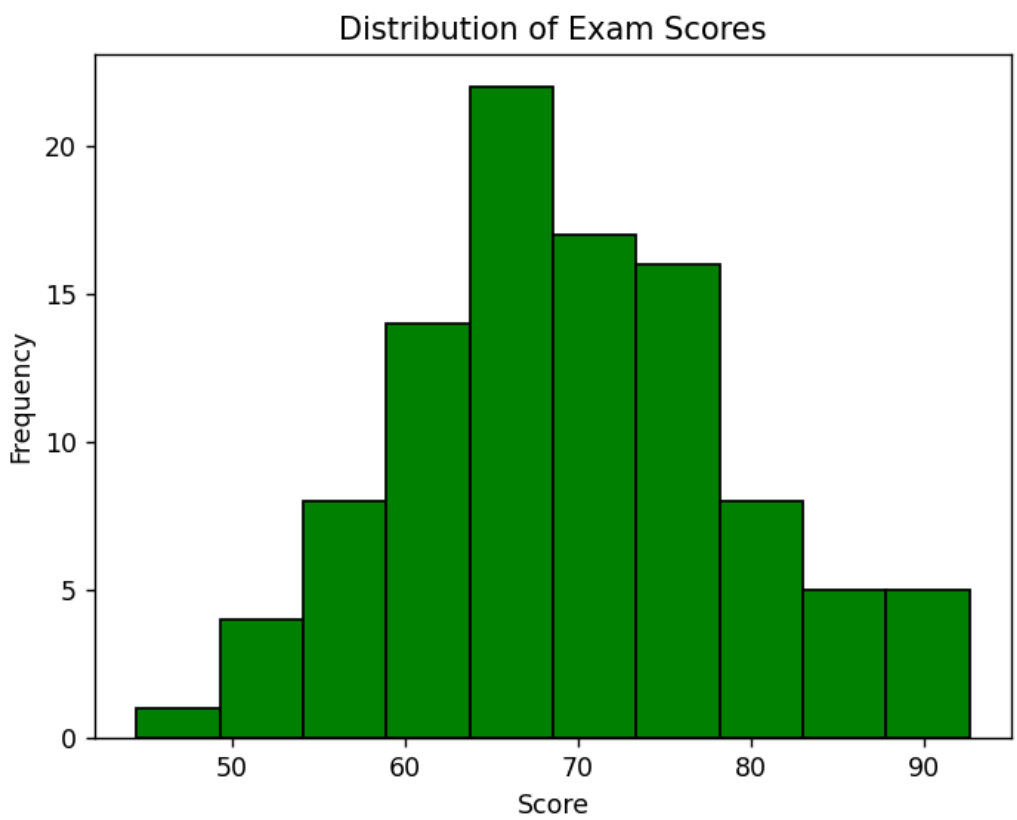
HISTOGRAM

Histogram is a type of bar graph where the graph is represented in groups

Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np
data = np.random.normal(70, 10, 100)
plt.hist(data, bins=10, color='green', edgecolor='black')
plt.title('Distribution of Exam Scores')
plt.xlabel('Score')
plt.ylabel('Frequency')
plt.show()
```

Output:



Description:

np.random.normal(70, 10, 100) generates 100 values from a normal distribution (simulating exam scores).

plt.hist() draws the histogram.

bins=10 splits the data into 10 equal-width intervals.

edgecolor='black' outlines each bar for better visibility.

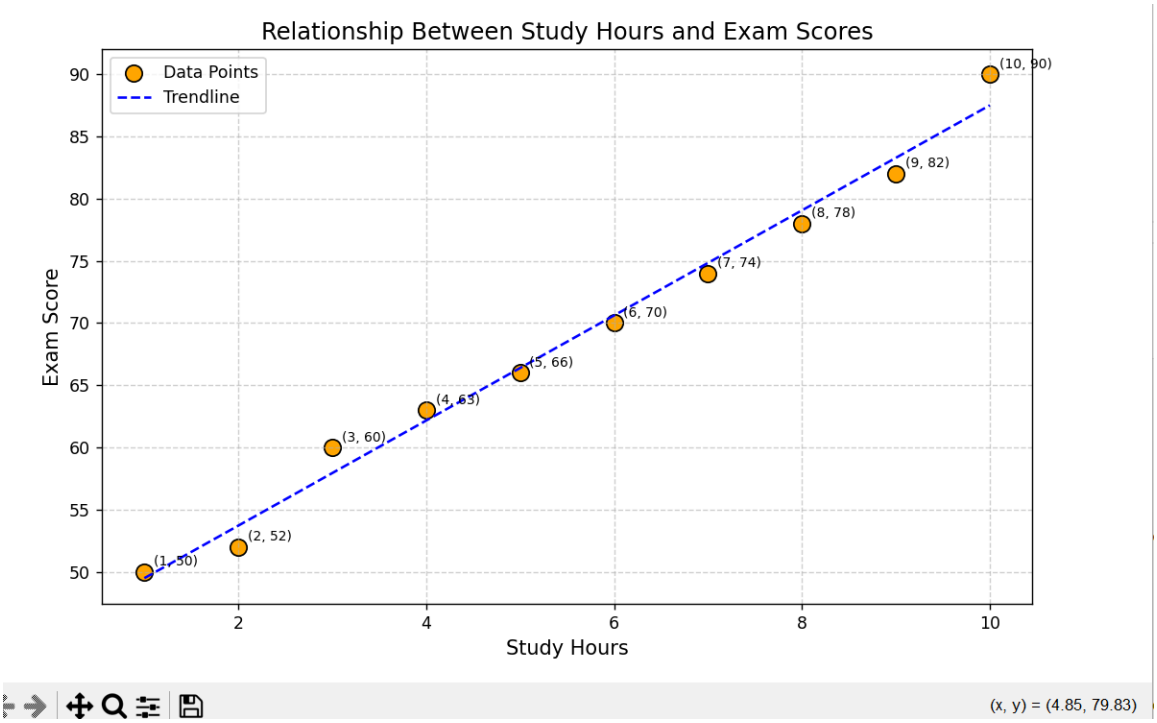
SCATTER PLOT

Scatter plot uses dots to depict the relationship between the data. scatter() is used for plotting scatter plot and scatter plot can be done for multiple datasets at the same time by differentiating it with colors. Legends help to achieve this difference.

Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np
study_hours = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
exam_scores = [50, 52, 60, 63, 66, 70, 74, 78, 82, 90]
x = np.array(study_hours)
y = np.array(exam_scores)
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='orange', s=100, edgecolor='black', label='Data Points')
plt.grid(True, linestyle='--', alpha=0.6)
plt.title('Relationship Between Study Hours and Exam Scores', fontsize=14)
plt.xlabel('Study Hours', fontsize=12)
plt.ylabel('Exam Score', fontsize=12)
m, b = np.polyfit(x, y, 1)
plt.plot(x, m*x + b, color='blue', linestyle='--', label='Trendline')
for i in range(len(x)):
    plt.text(x[i]+0.1, y[i]+0.5, f"({x[i]}, {y[i]})", fontsize=8)
plt.legend()
plt.savefig("scatter_plot_study_scores.png", dpi=300)
plt.show()
```

Output:



Description:

This code creates a scatter plot showing how study hours relate to exam scores. It plots each student's data point, adds a trendline to show the overall pattern, labels each point, and includes grid lines for clarity. The plot is saved as an image and displayed.

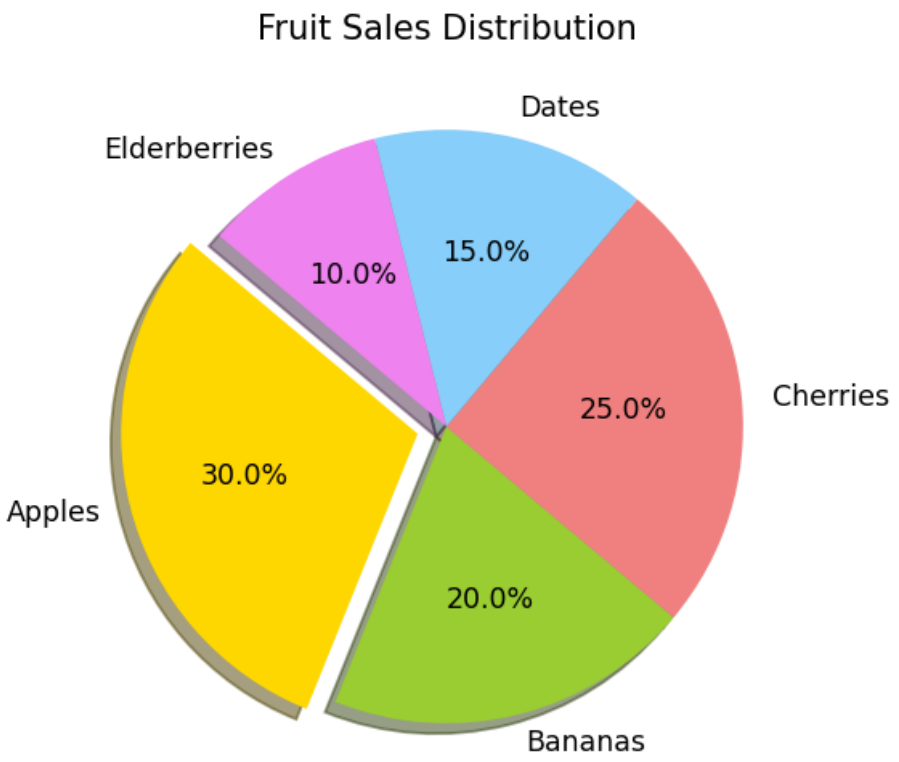
PIECHART

The input to a pie chart is a list of numerical values representing the sizes of each category (e.g., sizes = [30, 20, 25, 15, 10]).

Code snippet:

```
import matplotlib.pyplot as plt
sizes = [30, 20, 25, 15, 10]
labels = ['Apples', 'Bananas', 'Cherries', 'Dates', 'Elderberries']
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue', 'violet']
explode = (0.1, 0, 0, 0, 0)
plt.pie(sizes, labels=labels, colors=colors, explode=explode,
        autopct='%1.1f%%', shadow=True, startangle=140)
plt.title('Fruit Sales Distribution')
plt.show()
```

Output:



Description:

This code creates a pie chart that displays the distribution of fruit sales. Each slice size corresponds to the sales volume of a fruit, labeled accordingly. The first slice (Apples) is “exploded” to highlight it. Percentages are shown on each slice, and the chart has a shadow effect and a title.

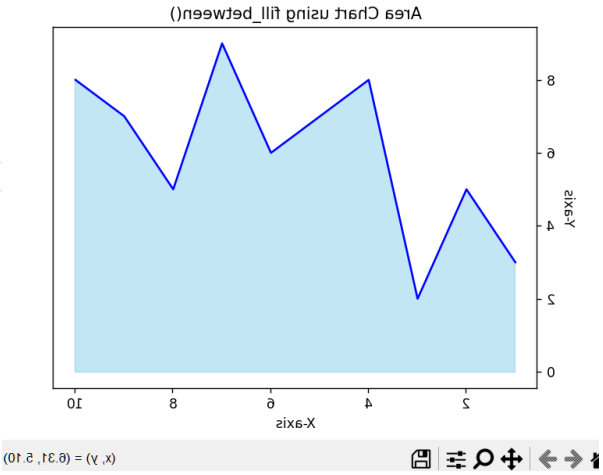
AREA CHART

In an area chart, the area between the line and the x-axis is filled or shaded to show volume or quantity over time or categories.

Code snippet:

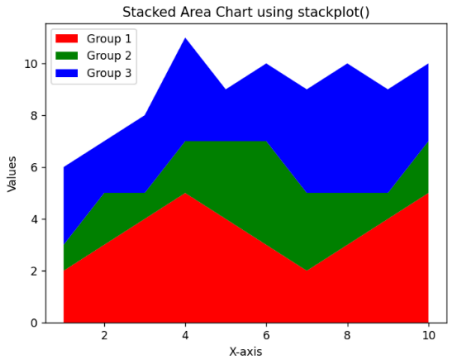
Using fill_between() function

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(1, 11)
y = np.array([3, 5, 2, 8, 7, 6, 9, 5, 7, 8]
plt.plot(x, y, color='blue')
plt.fill_between(x, y, color='skyblue', alpha=0.5)
plt.title('Area Chart using fill_between()')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



Using stackplot() function

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(1, 11)
y1 = np.array([2, 3, 4, 5, 4, 3, 2, 3, 4, 5])
y2 = np.array([1, 2, 1, 2, 3, 4, 3, 2, 1, 2])
y3 = np.array([3, 2, 3, 4, 2, 3, 4, 5, 4, 3])
plt.stackplot(x, y1, y2, y3, labels=['Group 1', 'Group 2', 'Group 3'], colors=['red', 'green', 'blue'])
plt.legend(loc='upper left')
plt.title('Stacked Area Chart using stackplot()')
plt.xlabel('X-axis')
plt.ylabel('Values')
plt.show()
```



Description:

n area chart fills the space under a line plot to show volume over time or categories.

- fill_between() fills the area under one line.
- stackplot() creates stacked areas for multiple groups with legends.

Bokeh

Bokeh is an interactive visualization library that allows the creation of sophisticated plots with high-performance interactivity. It provides both simple and complex visualizations, including interactive charts, dashboards, and web-based applications. Unlike static libraries like Matplotlib, Bokeh provides tools for building interactive plots and dashboards that can be embedded into web applications.

Unique Features of Bokeh:

1. **Interactivity**: Supports interactive features like hover, zoom, and panning.
2. **Flexibility**: Can generate a wide range of static, animated, and interactive visualizations.
3. **Output formats**: Outputs in HTML, PNG, SVG, and even server-based applications for real-time updates.
4. **Integration with web frameworks**: Easily integrates with web frameworks like Flask and Django.

Typical Use Cases:

1. **Data exploration**: Useful for building interactive plots during exploratory data analysis (EDA).
2. **Web dashboards**: Can be integrated into real-time web applications for displaying data to users.
3. **Interactive visualizations**: Perfect for building interactive data visualizations that respond to user input.

Types of Graphs in Bokeh

1. Line Plot
2. Scatter Plot
3. Bar Chart
4. Histogram
5. Pie Chart
6. Heatmap
7. Box Plot

Here are some sample codes for some of the graphs.

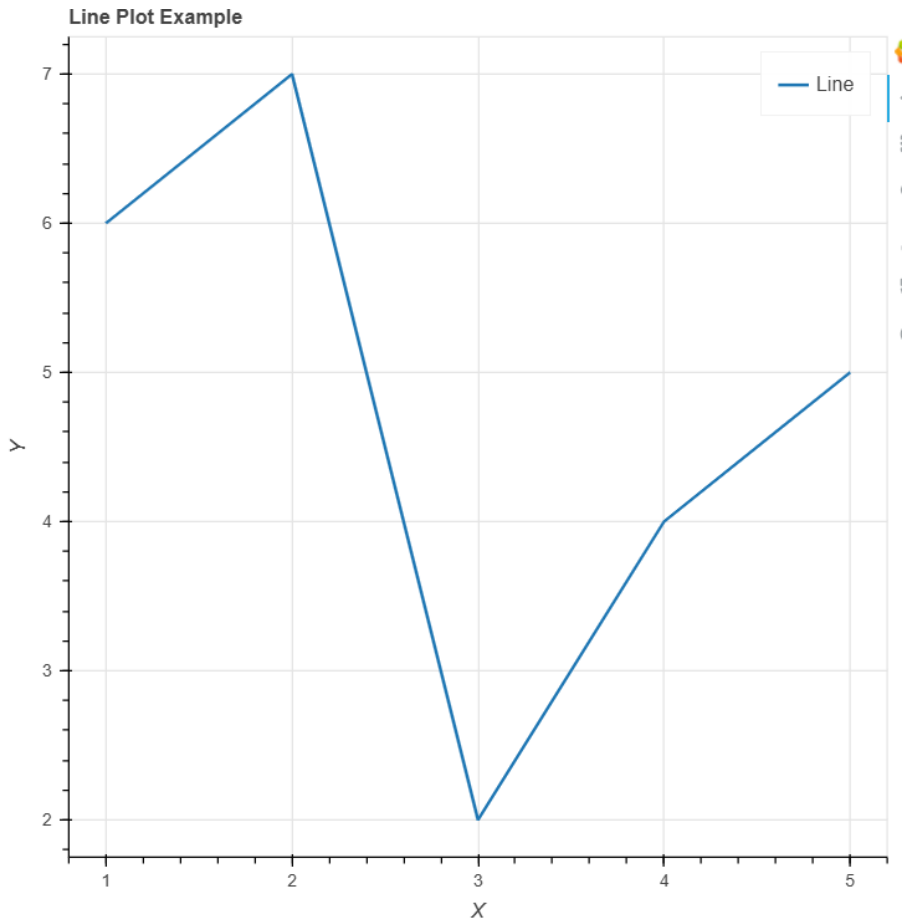
LINE PLOT:

A line chart is a type of chart that displays data points connected by straight line segments. It is used to represent continuous data over time or another variable, where each point represents a specific data value.

```
from bokeh.plotting import figure, show
from bokeh.io import output_file
x = [1, 2, 3, 4, 5]
y = [6, 7, 2, 4, 5]
p = figure(title="Line Plot Example", x_axis_label='X', y_axis_label='Y')
p.line(x, y, legend_label="Line", line_width=2)

output_file("line_plot.html")
show(p)
```

Output:



Description:

Line charts are typically used to display trends over time, such as stock prices, temperature changes, sales over a period, or any other data where the x-axis represents time or sequential data point.

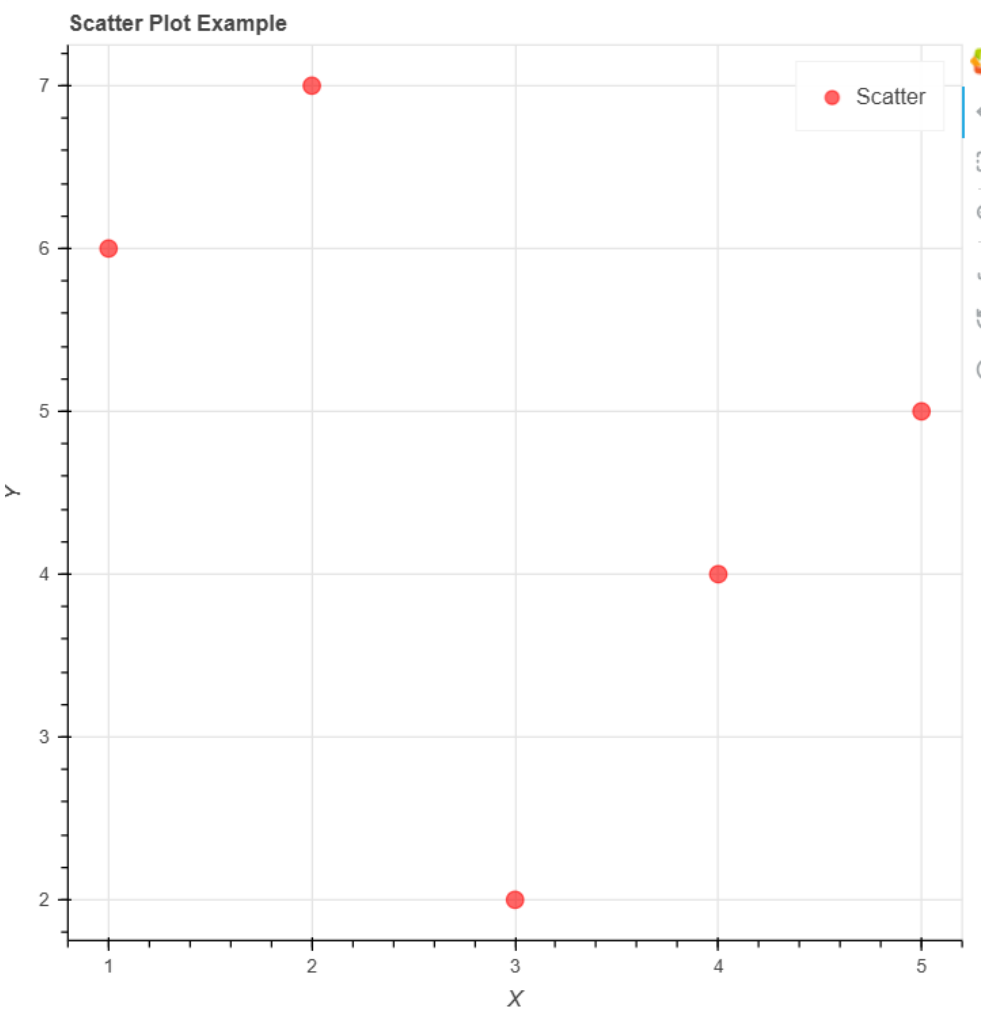
X-axis usually represents the time or continuous variable , Y-axis represents the measured variable ,

It is effective for visualizing data trends, making it easy to identify patterns such as increases, decreases, or cycles in the data.

SCATTER PLOT:

```
output_file("line_plot.html")
show(p)'''
from bokeh.plotting import figure, show
from bokeh.io import output_file
x = [1, 2, 3, 4, 5]
y = [6, 7, 2, 4, 5]
p = figure(title="Scatter Plot Example", x_axis_label='X', y_axis_label='Y')
p.scatter(x, y, legend_label="Scatter", size=10, color="red", alpha=0.6)
output_file("scatter_plot.html")
show(p)
```

Output:



Description:

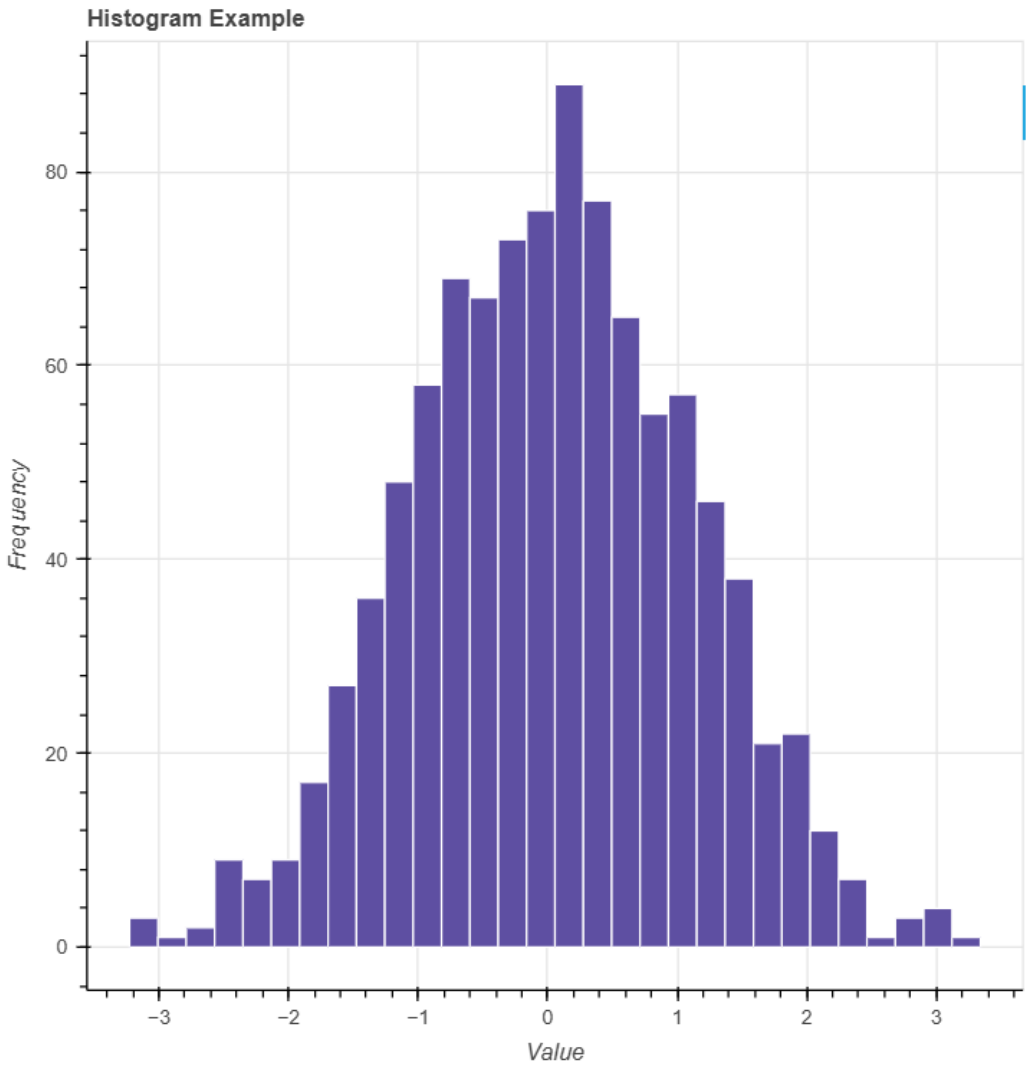
A scatter plot is a graphical representation of two variables, where each point on the plot corresponds to one data point with an x and y value. It is used to observe the relationship or correlation between two continuous variables.

HISTOGRAM

```python
import numpy as np
from bokeh.palettes import Spectral11
from bokeh.plotting import figure, show
from bokeh.io import output_file
from bokeh.transform import cumsum

data = np.random.randn(1000)
p = figure(title="Histogram Example", x_axis_label='Value', y_axis_label='Frequency')
hist, edges = np.histogram(data, bins=30)
p.quad(top=hist, bottom=0, left=edges[:-1], right=edges[1:], fill_color=Spectral11[0], line_color="white")
output_file("histogram.html")
show(p)
```

Output:



Description:

A histogram is a type of bar chart that represents the distribution of numerical data by grouping it into bins or intervals. The height of each bar shows the frequency of data points within each bin.

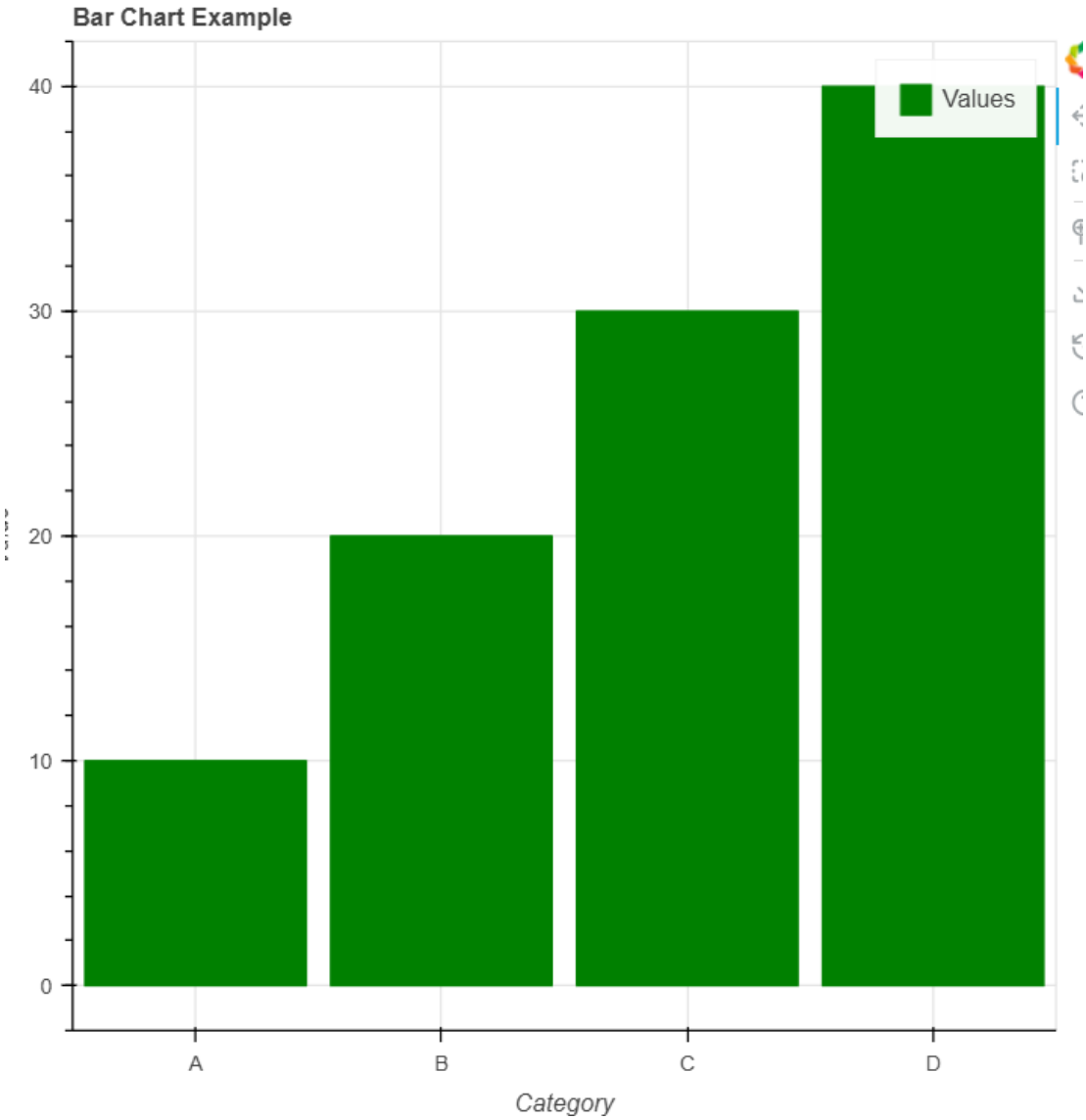
X-axis: Represents data intervals or bins , Y-axis: Represents the frequency (count) of data points within each bin.

The bars are adjacent, showing how data is distributed across different ranges.

BAR PLOT

```python
from bokeh.plotting import figure, show
from bokeh.io import output_file
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 30, 40]
p = figure(x_range=categories, title="Bar Chart Example", x_axis_label='Category', y_axis_label='Value')
p.vbar(x=categories, top=values, width=0.9, legend_label="Values", color="green")
output_file("bar_chart.html")
show(p)
```

Output:



Description:

A bar chart uses rectangular bars to represent the value of different categories. The length of each bar is proportional to the value it represents.

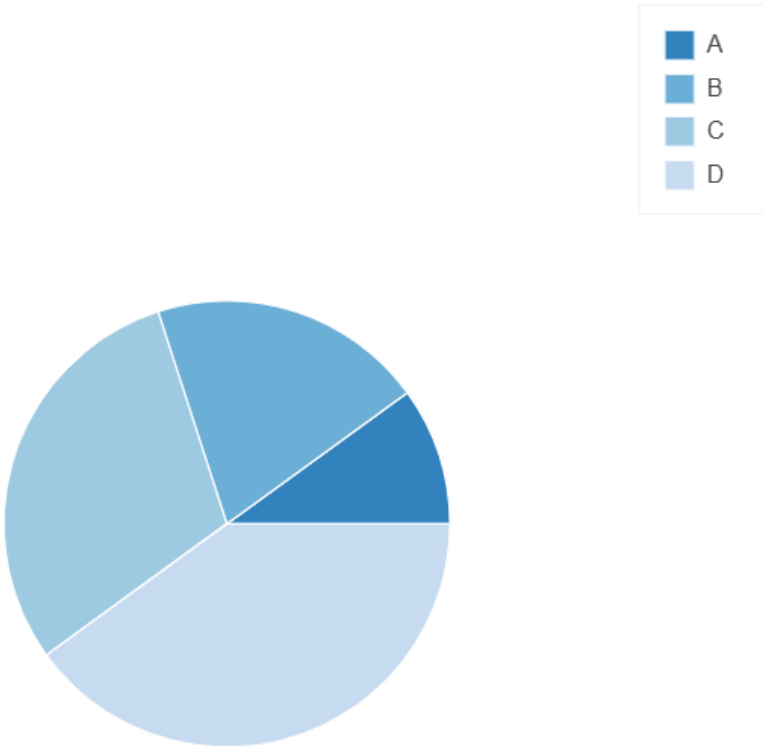
X-axis: Represents categories ,Y-axis: Represents the values for each category.

Bars can be vertical or horizontal, and their lengths show the magnitude of each category's value.

PIE CHART

```
from math import pi
import pandas as pd
from bokeh.palettes import Category20c
from bokeh.plotting import figure, show
from bokeh.io import output_file
from bokeh.transform import cumsum
data = pd.Series([10, 20, 30, 40], index=["A", "B", "C", "D"])
data = data.reset_index(name="value").rename(columns={"index": "category"})
data["angle"] = data["value"] / data["value"].sum() * 2 * pi
data["color"] = Category20c[len(data)]
p = figure(title="Pie Chart Example", toolbar_location=None, tools="hover", tooltips="@category: @value", x_range=(-1, 1))
p.wedge(x=0, y=1, radius=0.4,
        start_angle=cumsum('angle', include_zero=True), end_angle=cumsum('angle'),
        line_color="white", fill_color='color', legend_field="category", source=data)
p.axis.axis_label = None
p.axis.visible = False
p.grid.grid_line_color = None
output_file("pie_chart.html")
```

Output:



Description:

A pie chart is a circular chart divided into slices to illustrate numerical proportions. Each slice represents a category's contribution to the whole.

Used to show percentage or proportional data (e.g., market share by company, budget distribution), Ideal for displaying parts of a whole.

Each slice represents a category's percentage or value relative to the total.

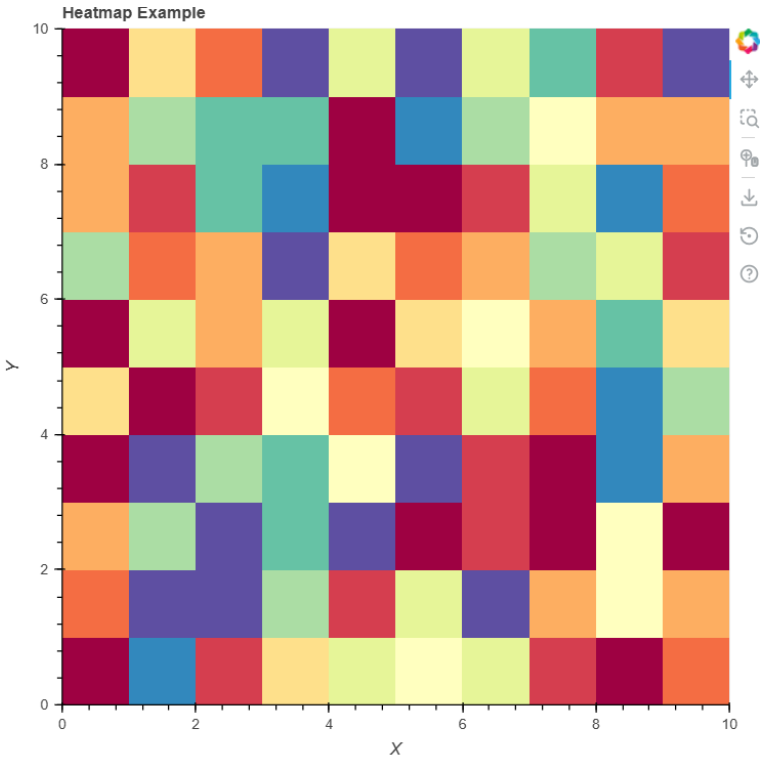
Easy to compare proportions visually. However, it's best for showing fewer categories to avoid clutter.

HEATMAP

A heatmap visualizes data through a matrix of colored cells, where each cell's color represents the value of the data point. Code snippet:

```
import numpy as np
from bokeh.plotting import figure, show
from bokeh.io import output_file
from bokeh.models import LinearColorMapper
from bokeh.palettes import Spectral11
data = np.random.rand(10, 10)
color_mapper = LinearColorMapper(palette=Spectral11, low=data.min(), high=data.max())
p = figure(title="Heatmap Example", x_axis_label="X", y_axis_label="Y",
           x_range=(0, 10), y_range=(0, 10))
p.image(image=[data], x=0, y=0, dw=10, dh=10, color_mapper=color_mapper)
output_file("heatmap.html")
show(p)
```

Output:



Description:

Data Generation: The code generates a 10x10 grid of random values between 0 and 1 using np.random.rand(10, 10).

Color Mapper: A LinearColorMapper is used to map the data values to colors from the Spectral11 palette, adjusting the colors based on the minimum and maximum data values.

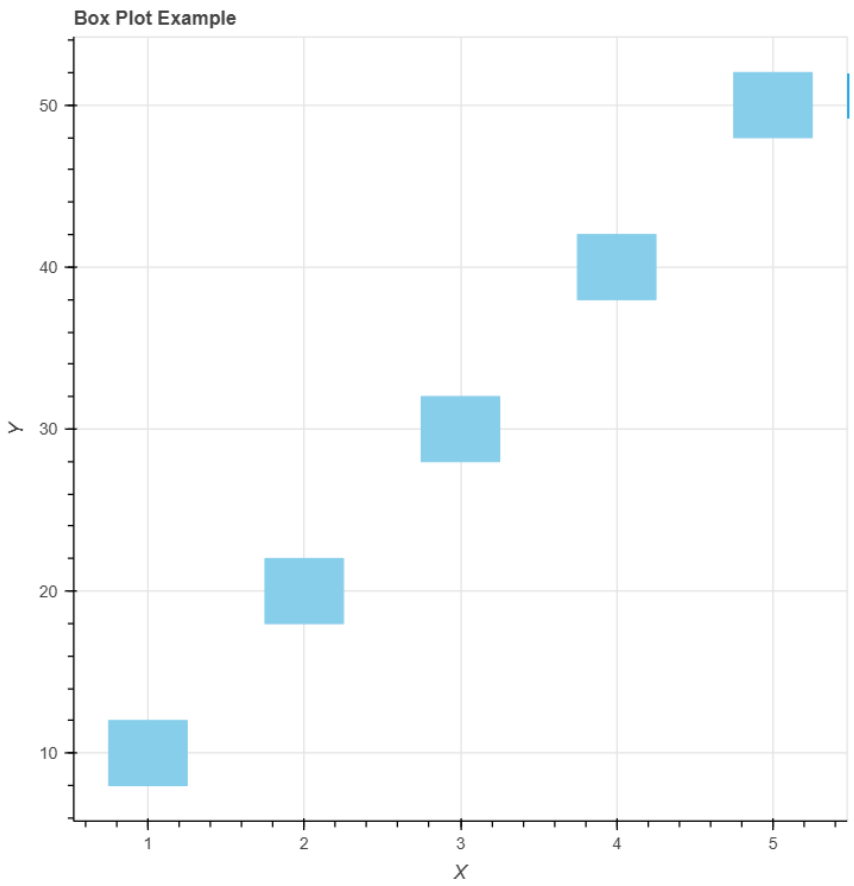
Plotting: The image() function adds the heatmap to the plot, where the data is shown as a 10x10 grid, Color Range: The color intensity (light to dark) represents the value range from the data, making it easy to identify higher and lower areas in the heatmap.

BOX PLOT

A box plot (or box-and-whisker plot) is used to show the distribution of a dataset, highlighting the median, quartiles, and outliers..

```
from bokeh.plotting import figure, show
from bokeh.io import output_file
data = {'x': [1, 2, 3, 4, 5], 'y': [10, 20, 30, 40, 50], 'upper': [12, 22, 32, 42, 52], 'lower': [8, 18, 28, 38, 48]}
p = figure(title="Box Plot Example", x_axis_label='X', y_axis_label='Y')
p.vbar(x=data['x'], top=data['upper'], bottom=data['lower'], width=0.5, color="skyblue")
output_file("box_plot.html")
show(p)
```

Output:



Description:

A box plot visually shows the distribution of data, highlighting its median, quartiles, and outliers.

Box: Represents the interquartile range (IQR) between the 25th and 75th percentiles.

Median: A line inside the box shows the median value (50th percentile).

Whiskers: Extend from the box to show the range of data, typically within 1.5 times the IQR.

Outliers: Data points outside the whiskers are considered outliers.

Useful for comparing data distributions and spotting outliers in a dataset.

COMPARISON OF MATPLOTLIB AND BOKEH

Matplotlib:

- **Core Library**:
  Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a low-level interface for creating plots with fine-grained control over every aspect of the figure.
- **Flexibility**:
  Matplotlib allows users to create any type of plot imaginable, from basic line plots and scatter plots to complex 3D plots, geographical maps, and more. Its flexibility is unmatched when it comes to producing highly customized plots.
- **Mature**:
  Matplotlib has been around for a long time (since 2003) and is a well-established library in the Python ecosystem. It has been adopted widely in the data science and scientific computing communities. Many other libraries (including Seaborn) are built on top of Matplotlib, which speaks to its importance and versatility.
- **Customization**:
  Matplotlib offers extensive customization options, allowing users to control every aspect of a plot, including colors, line styles, markers, fonts, annotations, tick marks, labels, and more. This fine-grained control makes it a powerful tool for creating highly tailored visualizations.
- **Integration**:
  Matplotlib can be easily integrated with other libraries and frameworks, such as NumPy, Pandas, SciPy, and others. This makes it a versatile and popular choice for data visualization in a wide variety of contexts, including scientific research, engineering, and machine learning.

Advantages of Matplotlib:

- **High Customization**: Full control over plot appearance and features.
- **Versatile Plot Types**: Supports almost any plot type you need (2D, 3D, polar, geographic, etc.).
- **Mature and Well-Documented**: Extensive community support and documentation.
- **Publication-Quality Plots**: Ideal for creating static, high-quality graphics for publications.

Use Case:

- Best for users who need fine control over plot details or are creating publication-quality static visualizations. Ideal for researchers and scientific visualization.

Bokeh:

- **Core Library**:
  Bokeh is a visualization library designed for creating interactive, web-based visualizations. Unlike Matplotlib, Bokeh is specifically tailored for generating dynamic, interactive plots that can be embedded in web applications.
- **Interactivity**:
  Bokeh shines in interactive visualizations. It has built-in support for zooming, panning,

hover tools, sliders, and real-time data updates. This makes Bokeh the go-to library for creating dashboards or web applications that require interactive exploration of data.

- **Ease of Use**:
  While Matplotlib offers extensive customization and control, Bokeh's API is more beginner-friendly when it comes to creating interactive plots. It has a more intuitive interface for adding interactive components (e.g., buttons, sliders, tooltips) to visualizations.
- **Performance with Large Datasets**:
  Bokeh is optimized for large datasets, especially in web applications. It can handle thousands (and even millions) of data points by using **WebGL** rendering, which ensures smooth performance even with large datasets.
- **Real-Time Interaction**:
  Bokeh allows for live updates and the ability to dynamically change the data displayed in the plot. This makes it ideal for real-time data streaming and visualization (e.g., monitoring dashboards).
- **Integration with Web Frameworks**:
  Since Bokeh is designed for web-based applications, it integrates seamlessly with web frameworks like Flask and Django. This makes it easy to embed interactive visualizations into web applications.

Advantages of Bokeh:

- **Interactivity**: Built for creating interactive plots with native support for tooltips, zooming, and panning.
- **Web Integration**: Ideal for embedding interactive plots in web pages or dashboards.
- **Real-Time Updates**: Can handle real-time data updates, perfect for dynamic or streaming data visualizations.
- **High Performance for Large Datasets**: Can efficiently handle large datasets with real-time interaction.

Use Case:

- Best for creating interactive visualizations for web applications, dashboards, and real-time data monitoring. Ideal for users who need to engage viewers with dynamic, interactive plots.